DEEP DIVE

The AI Coding Revolution by the Numbers

Three major reports landed in a single month — and together they tell a story the industry needs to hear

March 2026 | Engineering Digest | Special Edition

FOREWORD

In the space of three weeks in February and March 2026, three independent research efforts published their findings on AI in software engineering. The Pragmatic Engineer surveyed 906 engineers and found 95% using AI tools weekly. CircleCI analysed 28 million CI workflows and discovered that despite a 59% increase in development activity, delivery is actually slowing down. And Snyk's data shows that 48% of AI-generated code contains security vulnerabilities. None of these teams were coordinating. They were all looking at the same industry from different angles — and they arrived at the same place.

Each report on its own is interesting. Together, they paint a picture that should be uncomfortable for any engineering leader who thinks AI adoption means mission accomplished. The industry has embraced AI faster than it has adapted its testing, review, security, and delivery infrastructure to handle what AI produces. The result is a growing gap between code generation and code delivery — and engineering organisations are only beginning to reckon with it.

This issue is dedicated entirely to what the data says, what it means, and why the real work of the AI transition is just beginning.

EXECUTIVE SUMMARY

What the Data Says and Why It Matters

95%	59%	70.8%	48%
WEEKLY AI USAGE	THROUGHPUT INCREASE		AI CODE INSECURE

		MAIN BRANCH SUCCESS (5YR LOW)	

AI-assisted development is no longer an experiment. It is the default. The Pragmatic Engineer's survey of 906 professional software engineers found that 95% now use AI tools at least weekly, 75% use them for more than half their work, and 55% use autonomous agents regularly. In under eighteen months, agent usage has tripled from roughly 18% to 55%. The adoption curve is over. We are in the plateau of universal use.

But adoption is not the same as effectiveness. CircleCI's analysis of 28 million CI workflows tells the other side of the story: development activity is up 59% year-over-year — the largest increase in the report's seven-year history — but main branch throughput has actually declined by 7%. Main branch success rates have dropped to 70.8%, the lowest in more than five years. Recovery time is up 13%. Teams are writing more code than ever before and shipping less of it successfully.

Meanwhile, Snyk reports that 48% of AI-generated code contains security vulnerabilities, and Sonar estimates that 42% of all code is now AI-generated or AI-assisted. The arithmetic is not encouraging. Nearly half the code being produced by tools that nearly everyone uses is insecure, and the infrastructure to catch those issues before they reach production is not keeping pace.

Perhaps the most striking number in all three reports: only 5% of engineering teams have figured out how to actually ship at AI speed. The top 5% saw throughput gains of 97%. The median team saw 4%. The bottom quartile saw nothing. AI amplifies existing capability. It does not distribute it. The gap between the best teams and the rest is widening at a rate the industry has never seen, and the defining engineering challenge of 2026 is not whether to use AI — it's whether your organisation can deliver what AI produces.

THE ADOPTION STORY

How AI Became the Default

The Pragmatic Engineer's March 2026 survey is the most granular snapshot of AI tool adoption among professional software engineers published to date. 906 respondents, median experience of 11–15 years, primarily across Europe and the US, surveyed between January 27 and February 17, 2026. These are not junior developers experimenting with ChatGPT for the first time. These are experienced engineers describing how they actually work.

The headline number — 95% weekly usage — is striking but no longer surprising. What's more revealing is the depth. 75% of respondents use AI for more than half their daily work. 56% use it for 70% or more. And 55% now use AI agents regularly, up from roughly 18% just eighteen months ago. The shift from AI as a convenience to AI as a primary working method happened faster than most engineering leaders anticipated.

> *"95% of developers now use AI weekly — but only 29% trust it."*
>
> — The Pragmatic Engineer Survey, March 2026

The tool landscape has reshuffled dramatically. Claude Code, which launched in May 2025, reached the number one position within eight months. GitHub Copilot remains stable at around 46% usage, buoyed by enterprise procurement. Cursor has grown 35% in nine months. OpenAI's Codex has reached 60% of Cursor's usage despite being newer. 70% of engineers use two to four AI tools simultaneously — the single-tool era is already over.

Tool Usage by Company Size

At small companies, Claude Code dominates at 75% adoption, with Copilot trailing at 35%. At large enterprises (10,000+ employees), the picture inverts: Copilot leads at 56%, driven almost entirely by Microsoft procurement bundling and enterprise licensing agreements. This is not a reflection of engineer preference. It is a reflection of purchasing power.

The gap matters because experimentation rates are measurably higher where Claude Code is available. Engineers who choose their own tools are using different tools than engineers whose tools are chosen for them — and they report higher satisfaction and more aggressive adoption of agents.

Agent Usage by Role

Staff+ engineers are the heaviest agent users at 63.5%. This contradicts the assumption that senior engineers would be the most resistant to AI. Instead, they are the most aggressive adopters of the most autonomous tools. They are also, notably, the most likely to verify agent output before committing it.

Agent users are twice as excited about AI as non-agent users. They report that agents have fundamentally changed how they approach problems — not by writing code faster, but by changing which problems they choose to solve. The leverage is not speed. It's ambition.

The trust gap is the most psychologically interesting finding in the survey. Only 29% of developers say they trust AI-generated code. But 95% use it anyway. This is not cognitive dissonance — it's a rational response to a tool that is useful but unreliable. Engineers have adapted by building verification habits around AI output rather than trusting it implicitly. The question for organisations is whether those verification habits are systematic or ad hoc, and whether they scale.

Source: The Pragmatic Engineer

THE DELIVERY PARADOX

More Code, Less Delivery

CircleCI's 2026 State of Software Delivery report, published February 18, is the seventh annual analysis of CI/CD performance across their platform. This year's dataset: 28 million workflows across

thousands of engineering teams. It is, by volume, the largest empirical study of how software actually gets built and shipped. The finding that should concern every engineering leader: AI has supercharged code generation without a corresponding improvement in code delivery.

Average throughput increased 59% year-over-year — the largest jump since CircleCI began publishing the report in 2019. But that number conceals a bifurcation. Feature branch throughput (where AI-generated code lands first) rose 15% at the median. Main branch throughput (where code actually ships) fell 7%. More code is being written. Less of it is reaching production. The bottleneck has moved from generation to integration, and most organisations haven't noticed because they're measuring the wrong thing.

+97%	+4%	72m	1 in 20
TOP 5% THROUGHPUT	MEDIAN THROUGHPUT	RECOVERY TIME (+13%)	TEAMS SHIPPING AT AI SPEED

Main branch success rate has dropped to 70.8% — the lowest in more than five years of CircleCI's tracking. The benchmark is 90%. At a 70% success rate, a team running five changes per day faces 1.5 failures daily, compared to one failure every two days at the benchmark. Scale that to 500 daily changes across an organisation and you lose the equivalent of 12 full-time engineers to recovery work alone. That is not a rounding error. That is a hidden tax on every team that adopted AI generation without investing in validation infrastructure.

> *"AI amplifies existing delivery strengths. It doesn't distribute them evenly."*
>
> — CircleCI 2026 State of Software Delivery Report

Recovery time has risen to 72 minutes, up 13% year-over-year. Mid-sized companies (21–50 employees) are hit hardest: they show the lowest throughput gains and the longest recovery times at approximately three hours — four times longer than the smallest and largest organisations. These are companies large enough to have complex systems but too small to have dedicated platform or DevOps teams. They are in the messy middle, and AI is making it messier.

THE PERFORMANCE TIERS

> **Bottom Quartile** — No measurable throughput increase. AI adoption without infrastructure investment. These teams are generating AI code that fails in CI and costs more to recover from than it saved in writing.
>
> **Median Team** — +4% throughput, +15% feature branch activity. The code is being written, but it's piling up in feature branches and failing on merge. Main branch throughput is flat or declining.

> **Top 5%** — +97% overall throughput, +26% main branch, +85% feature branch. These teams invested in validation, testing, and CI infrastructure alongside AI generation. They are the proof that the delivery gap is solvable — and the evidence that most organisations haven't solved it.

Only 1 in 20 teams have figured it out. The top 5% saw overall throughput gains of 97%, with main branch throughput up 26% and feature branch throughput up 85%. These teams did not just adopt AI tools. They rebuilt their validation pipelines, invested in automated testing, and treated CI infrastructure as a first-class engineering concern. The lesson is not that AI doesn't work. It's that AI without delivery infrastructure is a net negative for most teams — and most teams don't know it yet.

Source: CircleCI

THE SECURITY GAP

48% of AI Code Is Insecure — and the Tools to Fix It Just Arrived

Snyk's ongoing analysis of AI-generated code arrives at a number that should stop every engineering leader mid-sentence: 48% of AI-generated code contains security vulnerabilities. Not style issues. Not lint warnings. Security vulnerabilities — the kind that end up in CVE databases and breach notifications. Time-to-exploit is shrinking and expected to accelerate by 50% by 2027 as AI-native components become embedded in more attack surfaces.

Sonar's State of Code survey puts the generation number in context: 42% of all code is now AI-generated or AI-assisted, and that figure is predicted to increase by more than half by 2027. The multiplication is uncomfortable. If 42% of your codebase is AI-generated and 48% of that is insecure, you are looking at roughly 20% of your entire codebase carrying vulnerabilities introduced by the tools you adopted to move faster. And that percentage is growing.

The industry's response has been fast — arguably faster than the response to the delivery problem. In the first week of March 2026 alone, three major AI security tools launched or expanded:

OpenAI Codex Security

Launched March 7. An AI security agent that scans repositories, identifies vulnerabilities, validates findings in a sandbox, and proposes fixes. In beta, it scanned 1.2 million commits and found 792 critical and 10,561 high-severity issues. Critical vulnerabilities appeared in under 0.1% of commits — a low percentage that becomes a large absolute number at scale. Available for Enterprise, Business, and Education tiers.

Anthropic Claude Code Security

Launched shortly before Codex Security. Scans codebases for vulnerabilities and suggests patches, integrated directly into the Claude Code workflow that engineers are already using for generation.

Cursor Security Automations

Launched March 5 as part of Cursor's broader automations platform. Triggers security reviews on every push to main, audits diffs for vulnerabilities, and posts high-risk findings to Slack. Cursor reports it has already caught multiple vulnerabilities and critical bugs internally.

The pattern is clear: the same companies building AI code generation are now building AI code security. The question is whether these tools can keep pace with the velocity of generation. When engineers are running three or four agents in parallel, producing code at a rate that has increased throughput 59% year-over-year, a security tool that runs after the fact is playing catch-up by design. The race between AI-generated vulnerabilities and AI-detected vulnerabilities is now the central tension in application security.

Sources: Snyk, Sonar, The Hacker News, TechCrunch

THE AGENT ERA

From Autocomplete to Autonomous

The evolution of AI coding tools follows a clear arc: autocomplete, then chat, then copilot, then agent. Each step gave the AI more context, more autonomy, and more surface area. Autocomplete suggested the next line. Chat answered questions. Copilots wrote functions. Agents write code, run tests, create pull requests, review their own work, and operate in parallel. The shift from copilot to agent happened in the last six months, and it is the most consequential transition in the sequence.

The numbers from February and March 2026 tell the story of an industry sprinting:

KEY DEVELOPMENTS — ONE MONTH

> **OpenAI Codex** — 1.6 million weekly active users, tripled with the release of GPT-5.3 Codex. Positioning itself as "the standard agent" for code generation, now complemented by Codex Security for vulnerability scanning.
>
> **Cursor Automations** — Always-on, event-driven agents that run hundreds of tasks per hour. Triggered by git events, Slack messages, or schedules. Security reviews on every push to main. This is not a copilot. It is an autonomous system that operates without a human in the loop.

GitHub Copilot — Coding agent with self-review capabilities, integrated security scanning, and cross-agent memory. The enterprise incumbent is adding agent capabilities to maintain its position with the 56% of large enterprises already locked in via procurement.

Stripe Benchmark — Claude Opus 4.5 scored 92% on real Stripe integration tasks. Not toy benchmarks. Real API calls, real payment flows, real error handling. This is the clearest signal yet that AI agents can handle production-grade work — at least in well-documented, well-structured domains.

> *"Claude Opus 4.5 scored 92% on real Stripe integration tasks — not toy benchmarks, but actual payment flows with real API calls and error handling."*
>
> — Stripe Engineering Blog, March 2026

The governance challenge is already visible in the Pragmatic Engineer data. At companies with 10,000+ employees, Copilot leads at 56% — driven by Microsoft procurement bundling, not by engineer preference. Engineers individually gravitate toward Claude Code, which leads at 75% in small companies where individuals choose their own tools. The gap between what engineers want to use and what organisations allow them to use is growing, and it has real consequences: teams with tool autonomy adopt agents faster, report higher satisfaction, and show stronger adoption of the practices that correlate with the top 5% in CircleCI's data.

The agent era is not coming. It arrived. 55% of engineers already use agents regularly. The question is no longer whether agents work — it's whether organisations can govern, secure, and integrate what agents produce.

Sources: The Pragmatic Engineer, TechCrunch, Stripe Engineering

WHAT THIS MEANS FOR ENGINEERING LEADERSHIP

The Four Problems Hiding in the Data

The three reports converge on a single conclusion: the hard part of the AI transition is not adoption. Adoption is done. The hard part is everything that comes after — and most organisations are not structured for it. Four problems are hiding in the data, and each one requires a different kind of attention.

The Measurement Problem

The People Problem

Traditional engineering metrics are becoming meaningless. Lines of code, PRs merged, story points completed — all of these are inflated by AI generation without a corresponding increase in delivered value. CircleCI's data makes the case clearly: you need to separately track feature branch throughput and main branch throughput. If feature branches are up and main branch is flat or declining, you have an integration problem, not a development problem. Most dashboards don't make this distinction. Most engineering leaders are looking at the wrong numbers.

The Security Problem

48% of AI-generated code is insecure. 42% of all code is AI-generated. The attack surface is growing faster than the security tooling can cover it. New AI security tools from OpenAI, Anthropic, and Cursor are arriving — but they are weeks old, operating at a fraction of the scale of the generation tools, and fundamentally reactive. The 29% trust figure from the Pragmatic Engineer survey is relevant here: if 71% of engineers don't trust AI code but ship it anyway, who is performing the security validation? Is it systematic, or is it dependent on individual engineer diligence?

Simon Willison, speaking at the Pragmatic Summit: "After two hours I'm done for the day. I'm mentally exhausted." Running three or four agents in parallel — reviewing their output, correcting their mistakes, integrating their work — is cognitively draining in ways that traditional coding is not. AI-assisted work is not less work. It is different work, at higher cognitive intensity, with a different fatigue curve. Engineers at different points on the adoption curve need different support, different expectations, and different measures of productivity. The organisations treating AI as a simple multiplier on existing capacity are going to burn out their best people.

The Delivery Problem

Only 5% of teams are shipping at AI speed. Main branch success rates are at a five-year low. Recovery times are rising. The bottleneck has moved from code generation to code integration, and mid-sized companies (21–50 employees) are hit hardest — too complex for simple workflows, too small for dedicated platform teams. The 12-FTE-equivalent cost of recovery at scale is not hypothetical. It is happening now, across thousands of organisations, and it is invisible to anyone who isn't tracking the right metrics.

None of these problems have obvious solutions, and this analysis is not going to pretend otherwise. What the data makes clear is that the AI transition is not a technology adoption problem. It is an organisational adaptation problem. The technology arrived. The adaptation hasn't. And the gap between the two is where engineering leadership will be tested in 2026.

Sources: The Pragmatic Engineer, CircleCI, Snyk, Waydev, Simon Willison

OTHER NEWS THIS WEEK

In Brief

AWS Middle East drone recovery continues — Day 16+, still no full restoration date published. AWS continues recommending workload migration to alternate regions. *The Register, CNBC*

Revolut secured full UK banking licence — FSCS-protected deposits for 13M customers, credit products coming, $75B valuation. *Bloomberg, CNBC*

Microsoft integrating Claude into Copilot Wave 3 — Anthropic's Cowork technology powering Agent 365, launching May. UK/EU tenants disabled by default. *eWeek, Microsoft Learn*

Open Banking valued at £43B potential for UK economy — EY analysis projects significant economic impact as adoption accelerates. *FinTech Global*

Node.js evolving release schedule — One major release per year from Node 27, every release becomes LTS. Simplifies upgrade planning. *nodejs.org*

ON THE RADAR

Coming Up

March 24	Node.js security releases (25.x/24.x/22.x/20.x)
April 19	Claude Haiku 3 model retirement
April 22	AWS Summit London
April 29	Zero Day Con, Dublin — Ireland's largest cybersecurity conference
April 30	Node.js 20 end of life
May	Microsoft Copilot Wave 3 / Agent 365 launch

RECOMMENDED READING

Sources & Further Reading

AI Tooling for Software Engineers in 2026	The Pragmatic Engineer
5 key takeaways from the 2026 State of Software Delivery	CircleCI
Can AI agents build real Stripe integrations?	Stripe Engineering
OpenAI Codex Security Scanned 1.2 Million Commits	The Hacker News

Cursor is rolling out a new kind of agentic coding tool	TechCrunch
The Engineering Leadership Blind Spot of 2026	Waydev
Why would anyone be an engineering manager in 2026?	LeadDev

Engineering Digest

March 2026 · Deep Dive · Special Edition

Next weekly edition: March 20, 2026